

# SYLTRA HEALTH — Technical Document

## How the platform is built: the layers, the engines, and access

Version 0.1, working draft. SYLTRA ONE. For the technical team, advisors and engineering review.

---

### 0. Overview

---

SYLTRA HEALTH is a software layer that connects health and home data, learns each person's normal pattern, and turns an important change into a check and then an alert to the most suitable person with the least necessary information. We build it on a few principles that settle most debates before they start: context before the number, the least necessary data, consent as a structure rather than a setting, every output explainable in one sentence, graduation before escalation, and Arabic as the language of design rather than of translation.

### 1. The five layers

---

We see the platform as five layers, each with inputs, an output and a readiness criterion, and we do not build a layer before the one it depends on is stable.

The **Health Link** layer gathers unified signals from sources the user permits, and is ready when at least two sources run steadily with a consent record. The **Home Awareness** layer reads movement, presence and environment reliably, without repeated false alarms. The **Personal Baseline** layer builds a baseline and a normal range for each user, stabilising after a learning period. The **Adaptive Environment** layer applies home settings tied to the occupant's state, with consent and instant reversibility. And the **Smart Response** layer verifies and alerts by graduation, with every alert carrying a presentable reason.

We build in order: the personal baseline first, then response, then home awareness, then the adaptive environment. The reason is simple: without a stable baseline, every alert becomes a false alarm, and a false alarm kills trust before it is built.

### 2. Data flow

---

Signals arrive from wearables and home sensors through an integration gateway that unifies their shape. They become organised derived signals, stored at the least necessary, and the engines then run on them on a schedule rather than when the app is opened. When warranted, the result passes to the response engine, which decides the appropriate level and composes an alert with a stored reason. The idea is that the sensitive logic is separate from the interface and runs in the background on a fixed schedule.

### 3. The personal-baseline engine

---

We write the pattern logic as pure functions, taking signals and a time passed as an argument, and returning a baseline or a "not yet mature" state. We set a learning period of at least two weeks, before which no deviation alert is issued and

the user is told plainly that the platform is still learning their pattern. We compute the baseline for each metric from a moving window, always compared to the individual rather than a group average. And we count a change as a deviation only when two independent signals fall outside the range within one window. Any period the user marks as travel or illness is excluded from building the baseline and from deviation detection. And any source dropout returns a technical "source gap," not a health deviation.

## **4. The response engine**

---

We also build it as pure functions, with a unit test named for each rule. The levels we build now are three: daily support, verification, and a trusted-person alert. The fourth level, authorised escalation, exists neither in the code nor in the database, and we defer it until agreements and approvals are complete. We hold to fixed rules: no move to a higher level before the lower is exhausted, except at the user's request; a wait time set in advance per user that does not change automatically; a default night silence; a weekly cap that halts non-critical alerts when exceeded; and every alert stores its reason, not the alert alone.

## **5. Medication adherence**

---

We compute the run-out date with a simple calculation from the dispense date, the units per pack and the daily dose, and set the alert a window ahead of it that allows for delivery time and a safety margin. If a daily dose confirmation is available for long enough, we replace the denominator with the observed consumption rate to raise accuracy. We clearly separate the pack-nearing-empty alert from the prescription-balance-exhausted alert, and group everything running out within a few days into one alert. And an alert becomes an order only with an explicit confirmation from the patient or an authorised caregiver, with a cancellation window before fulfilment.

## **6. Integrations**

---

Before accepting any data source we ask it three questions: is access authorised, is the data enough for context, and does its dropout break the user's experience. We build no logic that depends on a single source with no alternative. We display no entity's name or logo before a signed agreement, because appearing means partnership. Regulated medical devices are deferred to the Connected Care line and not admitted to the Wellness Safety line. And we design the gateway to read, from the health platforms' interfaces, the data types the user approves for each type on its own [5][6].

## **7. Consent and logs**

---

For us, consent is a database entity per scope, with a grant date and a revocation date, and every read or share path passes through an active-consent check that no query bypasses. We keep a sharing log the user can always see, and an audit log for every access to sensitive data. And deletion for us includes derived data, not only raw.

## **8. Access architecture for people of determination**

---

We treat access as a core presentation layer tested with every screen, not an add-on. The engineering idea is one, functional equivalence: we separate information from its presentation channel, so the same reading is delivered visually, audibly and haptically without duplicating logic for each channel. Every event in the system carries clear text and a reason code, so it turns automatically into a spoken reading, a light flash and vibration, or a written notice, according to the user's profile.

We offer access profiles the user chooses at setup and combines: motor, relying on voice and a single touch; hearing, turning every sound into a visual cue and vibration; visual, relying on screen reading, high contrast and a steady order; speech, allowing expression by touch or ready phrases; sensory, softening transitions and alerts; and cognitive, simplifying the interface to one step at a time. We build the interface to work fully with a screen reader, hold to the accessibility content standards [4], and provide one-tap scenes for the most frequent tasks. And any access setting that touches sharing or contact passes through the same consent and is logged.

## 9. Runtime architecture

---

We run the backend on Cloudflare Workers with a D1 database, because it already exists and has a low cold start. Evaluation for us is scheduled rather than on-demand, running through a daily job that updates baselines, computes run-out dates, fires due alerts, and detects disconnected sources. We write user-facing text in separate language files rather than in the code, Arabic first and right-to-left. And we keep a local-first principle in the design, so loss of internet does not stop local control where possible.

## 10. Technical security and privacy

---

We encrypt data in transit and at rest, manage secrets outside the code, and apply role-based access control with least privilege. We separate development and test environments from production, and use only synthetic data in development. And we hold to a documented lifecycle for health-sensitive software, so no rule is merged without a test, and the schema changes only through numbered migrations.

## 11. Reference standards

---

The standards and documents below are real. Confirm the latest edition and scope of applicability before any reliance.

1. IEC 62304. *Medical device software — Software life cycle processes*.
2. ISO/IEC 27001. *Information security management systems — Requirements*.
3. ISO 14971. *Medical devices — Application of risk management to medical devices*.
4. World Wide Web Consortium (W3C). *Web Content Accessibility Guidelines (WCAG) 2.1*. 2018.
5. Apple. *HealthKit Developer Documentation*.
6. Android. *Health Connect Developer Documentation*.
7. Cloudflare. *Workers and D1 Developer Documentation*.
8. HL7. *Fast Healthcare Interoperability Resources (FHIR)*.
9. IETF. *The OAuth 2.0 Authorization Framework (RFC 6749)*.

10. IMDRF SaMD Working Group. *Software as a Medical Device (SaMD): Key Definitions*. 2013.

## 12. Build roadmap

---

We build in phases, one at a time. We start with the foundation of a repository, migrations, tests and a health endpoint, then the consent structure and its logs so that no read path bypasses the consent check, then signal ingestion and the pattern engine, then the response engine at its first and second levels, then the trusted circle and the third level, then the medication-adherence line as the first tangible daily value. And we do not start a phase that depends on an unsettled decision before it is settled.

---

*End of document. Any principle we edit in the build document first, then here, then in the code.*